

update.ejs

cơ chế nạp dữ liệu (`<%=phone.xxx%>`) và form gửi về server (`action="/update/:id"`).

1. Cấu trúc tổng thể

```
<form action="/update/<%= phone.id %>" method="POST">
...
</form>
```

Giải thích:

- `action="/update/<%= phone.id %>"`
→ Đây là **đường dẫn mà form gửi dữ liệu đến** khi người dùng bấm nút `Save`.
 - Ví dụ nếu `phone.id = 3` thì URL thực tế là `/update/3`.
- `method="POST"`
→ Dữ liệu form được gửi bằng phương thức POST (phù hợp với lệnh `app.post("/update/:id")` trong `app.js`).

 *Kết nối với backend:*

Server trong `app.js` có đoạn:

```
app.post("/update/:id", (req, res) => { ... })
```

→ Khi bạn nhấn "Save", dữ liệu từ form này sẽ tự động đổ vào `req.body`, còn `id` nằm trong `req.params`.

2. Phần input và cú pháp EJS

Ví dụ:

```
<input type="text" class="form-control" id="name" name="name" value="<%= phone.name %>" />
```

Giải thích:

- `value="<%= phone.name %>"` nghĩa là:
 - EJS sẽ **chèn giá trị thật của phone.name** (từ server gửi xuống) vào thẻ input.
 - Nếu `phone.name = "iPhone 15 Pro"`, HTML thực tế khi hiển thị sẽ là:

```
<input ... value="iPhone 15 Pro">
```

- Nhờ vậy, người dùng thấy sẵn dữ liệu cũ trong ô nhập để chỉnh sửa.

Tương tự cho:

```
value="<%= phone.brand %>"
```

```
value="<%= phone.price %>"
```

3. Các thành phần Bootstrap

- `class="form-control"` : giúp input có giao diện chuẩn Bootstrap.
- `col-md-6` : mỗi ô chiếm ½ hàng (trên màn hình $\geq 768\text{px}$).
- `form-label` : tạo khoảng cách và style cho label.
- `btn btn-primary` / `btn-secondary` : 2 kiểu nút khác nhau (nút chính và phụ).
- `bg-light` : nền sáng, làm giao diện nhẹ nhàng.

 *Bootstrap chỉ là phần giao diện — không ảnh hưởng tới logic EJS.*

4. Cách EJS nạp dữ liệu vào view

Từ `app.js` :

```
app.get("/update/:id", (req, res) => {  
  const { id } = req.params;  
  let phone = phones.find(item => item.id == id)  
  res.render("update", { phone })  
})
```

Khi bạn truy cập `/update/3` :

1. Express lấy `id=3` từ URL.
2. Tìm ra object tương ứng trong mảng `phones` .
→ `{ id: 3, name: "Google Pixel 8 Pro", brand: "Google", price: 900 }`
3. Gửi đối tượng này vào view EJS qua `{ phone }` .
4. EJS thay toàn bộ `<%= phone.xxx %>` bằng giá trị thật.

5. Khi bấm **Save**

- Trình duyệt gửi dữ liệu về `/update/3` qua POST.

- Server xử lý:

```
const { id } = req.params;
const data = req.body;
const index = phones.findIndex(item => item.id === id);
if (index !== -1) {
  phones[index] = { ...phones[index], ...data, id: Number(id) };
}
res.redirect("/");
```

- Nghĩa là:
 - Lấy dữ liệu mới (`req.body.name` , `req.body.brand` , `req.body.price`)
 - Ghi đè vào sản phẩm cũ có `id=3`
 - Quay lại trang danh sách (`/`)

Tóm tắt cơ chế

Bước	Diễn giải
1	Người dùng bấm nút Update tại <code>/update/:id</code>
2	Server render <code>update.ejs</code> , truyền <code>{phone}</code>
3	EJS hiển thị sẵn dữ liệu cũ vào form
4	Người dùng chỉnh và bấm Save
5	Dữ liệu gửi về <code>/update/:id</code> qua POST
6	Server cập nhật mảng và redirect về danh sách

Mẹo mở rộng:

- Nếu bạn muốn **ô Email chỉ đọc**, thêm thuộc tính:

```
<input type="text" name="email" value="<%= phone.email %>" readonly>
```

- Nếu muốn **định dạng số tiền**:

```
<td>$<%= phone.price.toLocaleString() %></td>
```

- Nếu form có lỗi nhập liệu, bạn có thể hiển thị báo lỗi bằng:

```
<% if(error){ %>
<div class="alert alert-danger"><%= error %></div>
```

<% } %>
